Implementation of a Java-Based Virtual Classroom System

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0927- Programming Language in Java

to the award of the degree of

BACHELOR OF ENGINEERING

Computer Science and Engineering

Submitted by

Ch.Naga Sai Srikanth(192311159)

Under the Supervision of

Dr.Pushpalatha S

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Implementation of a Java-Based Virtual Classroom System** " has been carried out by **Ch.Naga Sai Srikanth(192311159)** under the supervision of Dr.Pushpalatha S and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr Charlyn Pushpalatha

Professor

Department of CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.Pushpalatha S

Professor

Department of CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, **Ch.Naga Sai Srikanth(192311159)** of the Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Implementation of a Java-Based Virtual Classroom System" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Chennai

Date:

Signature of the Students with Names

Ch.Naga Sai Srikanth(192311159)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student / Registration No.	**Specific Responsibilities**	**Design & Development**	**Testing & Analysis**	**Report Contribution**	**Approx. %**
N.Navya Sri Sasavi 192411204	Requirements, Module 1, integration	User Management and Authentication	Login, authorization and security tests	Chapters 1–3; Appendix C	[25%]
B. Lahari sri 192421421	Module 2 and content workflow	Live classroom, course/resource management	Session and content tests	Chapter 4; Appendix B/E	[25%]
Challa Priyadarshini 192311118	Module 3 and analytics	Assignments, assessments, reports	Grading, report accuracy and usability tests	Chapters 4–5	[25%]

Ch. Naga Sai Srikanth 192311159	Module 4 and documentation	Discussion, group management, final documentation	Moderation, privacy and integration tests	References and appendices	[25%]
Total					100%

ABSTRACT

Virtual classroom platforms must coordinate identity, synchronous teaching, learning resources, assessment, feedback, reporting, and peer interaction without sacrificing security, usability, or maintainability. The engineering problem addressed in this project is the design and implementation of a Java-based system that integrates these functions into a coherent, role-aware application for administrators, faculty members, and students. The need is significant because fragmented tools create duplicated data, inconsistent access permissions, delayed feedback, and limited visibility into learning progress. UNESCO guidance emphasizes selecting technologies according to connectivity and digital readiness, ensuring inclusion, protecting privacy, supporting human interaction, and monitoring learning progress .

The proposed solution is a modular virtual classroom system organized into four functional modules: User Management and Authentication; Live Classroom and Content Management; Assignment, Assessment and Reports; and Reports and Group Discussion. A layered architecture separates the presentation, service, persistence, and database concerns. Java is selected as the implementation platform because Java SE supports portable, secure, server-oriented application development The design applies role-based authorization, server-side validation, parameterized persistence operations, audit-oriented records, and accessibility-conscious interfaces. Security requirements are informed by the OWASP Application Security Verification Standard , authentication decisions are aligned with NIST Digital Identity Guidelines, and accessibility requirements are framed using WCAG 2.2's perceivable, operable, understandable, and robust principles.

The implementation is evaluated through a requirements traceability matrix, module-level functional tests, negative authorization tests, data-integrity checks, usability checks, and integration scenarios covering the complete learning cycle. Because project-specific performance measurements and screenshots depend on the final deployed build, this report distinguishes engineering targets from achieved values and provides spaces for verified evidence. The principal outcome is a maintainable prototype architecture that brings classroom delivery, content access, assessment, analytics, and collaborative discussion into one controlled system.

Keywords: virtual classroom; Java application; role-based access control; e-learning; assessment analytics; group discussion

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

A virtual classroom is a socio-technical system in which teaching, learning, communication, assessment, and evidence of participation are mediated by software and network services. Unlike a static content repository, a classroom system must coordinate several stateful processes: a user must be identified before accessing a course; an instructor must create and schedule learning activities; a student must receive resources, submit work, participate in discussions, and obtain feedback; and an administrator must maintain governance and reporting. The system therefore combines information management, authentication, workflow control, user-interface design, database consistency, and security engineering.

The proposed project implements these concerns in a Java-based application. The platform is designed around four modules. Module 1 provides registration, login, role assignment, profile management, and access control. Module 2 provides course spaces, live-class scheduling or session entry, announcements, and content-resource management. Module 3 manages assignments, submissions, assessment records, grading, and reports. Module 4 supports discussion groups, moderation, participation records, and consolidated reports. The modules share a common identity and persistence layer so that a learner's activities can be connected without repeatedly collecting the same information.

1.2 Need for the Project

The project is needed because educational activities are often distributed across independent communication, storage, meeting, and assessment tools. This fragmentation increases operational effort and makes it difficult to maintain a reliable relationship between a user, a course, a submission, a grade, and a participation record. Students may miss announcements or deadlines, faculty may manually reconcile attendance and assessment data, and administrators may lack a consistent view of course activity. UNESCO recommends selecting digital-learning tools according to local connectivity, device access, and digital skills; it also highlights inclusion, privacy, human interaction, planned schedules, and learning monitoring as design concerns [1]. These concerns translate directly into engineering requirements for the proposed system.

The affected stakeholders are students, faculty, administrators, academic coordinators, and support personnel. Students need predictable access, clear deadlines, feedback, and respectful communication. Faculty need efficient resource publishing, assessment workflows, and evidencebased reports. Administrators need controlled account management, auditability, and system-level summaries. The engineering significance lies in balancing conflicting requirements: the system must be simple enough for routine academic use, but sufficiently secure to protect personal and assessment data; responsive enough for ordinary classroom activity, but structured enough to maintain data consistency; and extensible enough to support future integrations without making the prototype unnecessarily complex.

1.3 Problem Statement

Design and implement a Java-based virtual classroom system that integrates authenticated, roleaware access; live classroom and learning-content workflows; assignment and assessment management; academic reporting; and moderated group discussion, while meeting realistic constraints related to usability, privacy, security, maintainability, internet variability, development time, and limited project resources. The system shall preserve data integrity across modules, prevent unauthorized access to academic records, provide traceable workflows for teaching and learning activities, and support verification through measurable functional and non-functional tests.

1.4 Project Aim

To design, implement, and evaluate a modular Java-based virtual classroom system that provides secure access, organized learning delivery, assessment support, academic reporting, and collaborative discussion in one integrated platform.

1.5 Project Objectives

To elicit and document stakeholder, functional, security, accessibility, and performance requirements for a virtual classroom system.

To design a layered Java architecture and relational data model that separates interface, business logic, persistence, and security concerns.

To implement Module 1 for user management, authentication, role-based authorization, and profile governance.

To implement Module 2 for live classroom scheduling or session access, course organization, announcements, and content-resource management.

To implement Module 3 for assignment creation, submission handling, assessment, grading, and learner-performance reports.

To implement Module 4 for group discussion, moderation, participation tracking, and consolidated reporting.

To verify the system through functional, integration, security, usability, and data-integrity testing using predefined acceptance criteria.

To evaluate engineering trade-offs, sustainability, ethics, safety, risk, security, project management, and future scalability.

1.6 Scope

Included scope comprises a browser-accessible application interface; administrator, faculty, and student roles; secure login and logout; role-aware navigation; course and classroom records; learning resources and announcements; assignment and submission workflows; assessment and grade records; reports for academic progress and participation; discussion groups with moderation; relational persistence; input validation; audit-oriented timestamps; and a testable modular codebase. The scope also includes system documentation, requirements traceability, a risk register, a budget estimate, and a contribution statement.

Excluded scope comprises production-grade video transcoding, a global content-delivery network, biometric identity verification, institution-wide single sign-on, automated proctoring, payment processing, advanced adaptive-learning algorithms, and unrestricted public deployment. Live teaching may be represented through scheduled sessions and a session-entry workflow or through an approved meeting-service integration; the exact media transport should be documented separately if implemented. Assumptions include availability of a modern browser, a functioning network connection for synchronous activity, an institutional administrator who can manage roles, and a relational database accessible to the application. The boundary condition is that the system is a capstone prototype and not a certified production platform.

1.7 Expected Engineering Outcome

The expected outcome is a working prototype and documented software architecture for an integrated virtual classroom system. The deliverable includes a Java application, database schema, module-level interfaces, test evidence, user-role workflows, reports, and a maintainable repository structure. The engineering contribution is not merely a collection of screens; it is an integrated information system .

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review used a focused engineering search of official guidance, standards documentation, platform documentation, and representative categories of existing educational systems. Sources were selected for authority, technical relevance, and direct applicability to the project. UNESCO guidance was used to frame educational and inclusion concerns; Oracle Java documentation was used to justify the implementation platform; NIST SP 800-63B was used to inform digital authentication decisions; OWASP ASVS was used to structure application-security verification; W3C WCAG 2.2 was used for accessibility; and ISO/IEC 25010 was considered as a quality-evaluation reference. The review also examined common platform patterns such as learning-management systems, videoconferencing tools, content repositories, and discussion forums rather than treating any single commercial product as the complete design target.

2.2 Review of Existing Work

Distance-learning guidance treats technology as a means of maintaining educational continuity, but it also warns that solutions must match power, connectivity, device access, and digital skills. It recommends inclusion, privacy and security, teacher support, limited platform fragmentation, learning-process monitoring, and the creation of communities [1]. These principles support an integrated platform in which students do not need to move between several unrelated systems for every learning activity.

Java is suitable for a capstone implementation because Java SE supports development and deployment on desktops and servers and provides portability, performance, versatility, and security characteristics relevant to server-oriented applications [2]. A layered Java design also makes it possible to isolate authentication, classroom workflows, assessment logic, reporting queries, and discussion moderation. This separation reduces the likelihood that interface code will directly manipulate sensitive persistence operations.

For identity, NIST SP 800-63B defines authentication as establishing that a claimant controls an authenticator associated with a subscriber account and describes authentication assurance levels, authenticator management, session management, and privacy considerations [4]. The capstone

prototype does not claim formal conformance to a government assurance level; instead, it adopts the applicable engineering principles: password secrets should not be stored in plaintext, sessions should be invalidated on logout, role permissions should be checked server-side, and stronger authentication can be added when institutional risk requires it.

OWASP ASVS provides a basis for testing technical security controls and a list of requirements for secure development, including protection against common classes of vulnerability such as cross-site scripting and SQL injection [3]. The proposed design therefore treats validation, output encoding, parameterized database access, authorization checks, secure session handling, error management, and security-focused testing as requirements rather than optional refinements.

WCAG 2.2 provides technology-independent, testable success criteria organized under four principles: perceivable, operable, understandable, and robust [5]. For this project, the practical implications include meaningful labels, keyboard-accessible controls, sufficient contrast, clear error messages, visible focus, responsive layouts, and alternatives for information that would otherwise depend exclusively on audio or video. Accessibility is especially relevant in a virtual classroom because learning barriers can be amplified when content is delivered through a single interaction mode.

2.3 Comparative Analysis

Existing Approach	**Advantages**	**Limitations**	**Relevance to Proposed Work**
Video-conferencingonly tools	Strong synchronous communication; mature audio/video features.	Weak assignment lifecycle, gradebook, structured reports, and persistent academic records.	Module 2 may integrate or link to a session service, but the proposed system owns course and learning workflows.
Learning-management systems	Good course organization, assessment, content, and reporting.	May be complex to configure; live interaction and lightweight discussion can be fragmented or plugin-dependent.	The four-module design adopts the integrated workflow while keeping the capstone scope manageable.

Content repositories	Simple file storage and sharing; easy to understand.	Limited identity context, assessment, feedback, discussion, and progress reporting.	Module 2 treats content as courselinked resources with access control and metadata.
Messaging and discussion platforms	Fast communication and community formation.	Poor a cademic traceability, moderation governance, assessment and integration.	Module 4 adds structured groups, moderation, participation records, and report linkage.
Custom Java prototype	Tailored requirements, transparent architecture, controlled data model, and educational value.	Requires careful testing, security, and deployment, media maintenance; scalability is l in a prototype.	Selected approach for demonstrating integrated engineering design and modular implementation.

2.4 Research / Engineering Gap

The unresolved engineering gap is not the absence of individual tools; it is the dependable integration of identity, classroom activity, content, assessment, reports, and discussion under one coherent data and authorization model. Many solutions optimize a particular function, whereas a capstone implementation must demonstrate how subsystems interact and how requirements are verified. A further gap is the translation of high-level principles—privacy, accessibility, inclusion, and security—into observable design decisions and test cases.

2.5 Proposed Contribution

This project contributes a modular Java-based prototype and a documented engineering process. Its contribution is the explicit mapping of four modules to stakeholder needs, data entities, accesscontrol rules, test cases, risk controls, and outcome evidence. It also contributes a realistic boundary: synchronous media is treated as a controlled session workflow rather than an unverified .

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Students	Authenticate securely; view enrolled courses; access resources; attend or enter live sessions; submit assignments; view feedback and reports; participate in moderated groups.
Faculty	Create and manage courses; schedule sessions; publish content; create assignments; evaluate submissions; generate reports; moderate discussions.
Administrator	Manage accounts, roles, course membership, system configuration, audit-oriented records, and aggregate reports.
Academic coordinator	Review participation, assessment status, and course-level summaries without editing individual academic evidence unless authorized.
Institution / support	Receive maintainable documentation, predictable error handling, backup guidance, and security-relevant logs.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target

FR-01: authenticate users and create a role-aware session	Functional	Valid credentials succeed; invalid credentials fail;
		unauthorized role routes are denied.
FR-02: manage users and course membership	Functional	Admin can create, update, deactivate, and assign roles subject to validation.
FR-03: manage live-class sessions and course content	Functional	Faculty can publish a session/resource; enrolled students can view permitted items.
FR-04: manage assignments, submissions, grading, and feedback	Functional	A submission retains student, assignment, timestamp, status, grade, and feedback.
FR-05: provide reports and discussion groups	Functional	Authorized users can view relevant report scopes and moderated discussion threads.
NFR-01: security	Non-Functional	Server-side authorization, password hashing, validation, safe persistence, and session invalidation are demonstrated.
NFR-02: usability and accessibility	Non-Functional	Core workflows are keyboard navigable, labeled, readable, and tested with representative users.
NFR-03: reliability and integrity	Non-Functional	Critical transactions preserve referential relationships and return controlled errors.

NFR-04: maintainability	Non-Functional	Modules are separated into clear services/repositories with documented interfaces and tests.
NFR-05: performance target	Non-Functional	In the capstone test environment, ordinary
		page/API interactions target a median response below 2 seconds, excluding external media startup.

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
OWASP ASVS 5.0.0	Use a structured basis for web-application security verification [3].	Authentication, authorization, injection prevention, output encoding, session management, error handling, and security test cases are mapped to the relevant workflows.
NIST SP 800-63B Rev. 4	Provide sound digitalauthentication and session principles [4].	Password policy, authenticator handling, session invalidation, and optional future step-up authentication are documented; formal AAL certification is not claimed.
W3C WCAG 2.2	Make web content perceivable, operable, understandable, and robust [5].	Labels, focus order, contrast, error messaging, keyboard operation, and alternatives for media are included in design and verification.

ISO/IEC 25010:2023	Use a quality model for requirements and evaluation [6].	Functional suitability, performance efficiency, usability, reliability, security, maintainability, and portability guide the test plan.
Data-protection principles	Collect only needed education	Role scope, data minimization,
	and identity data; protect and govern access.	consent/notice placeholders, retention decisions, and restricted reports are included.

Technical constraints include finite development time, a capstone-sized team, a prototype deployment environment, dependence on browser and network availability, and the need to avoid claiming production-scale video reliability without measured evidence. Economic constraints favor open-source Java tooling and a relational database. Ethical constraints require honest reporting of test results, explicit acknowledgement of external and AI-assisted resources, protection of student records, and non-discriminatory access. Health and safety concerns are primarily digital well-being and accessibility: the platform should avoid unnecessary screen complexity, provide clear session information, and not make essential learning evidence available only through a single sensory channel.

3.3 Design Specifications

Parameter	**Target / Requirement**	**Unit**	**Priority**
Supported roles	Administrator, faculty, student, coordinator	roles	High
Core modules	Four modules with shared identity and persistence	modules	High
Password storage	Salted, adaptive oneway password hash; never plaintext	implementation rule	High

Authorization	Server-side RBAC on every protected operation	check	High
Data integrity	Foreign keys or equivalent application constraints for core entities	constraint	High
Accessibility	Keyboard operation, labels, focus visibility, readable contrast, clear errors	WCAG-aligned checks	Medium
Response target	Median ordinary interaction below 2 seconds in test environment	seconds	Medium
Auditability	Created/updated timestamps and actor context for critical records	record fields	Medium
Backup readiness	Documented export/backup and restore procedure for project database	procedure	Medium

3.4 Alternative Solutions & Evaluation

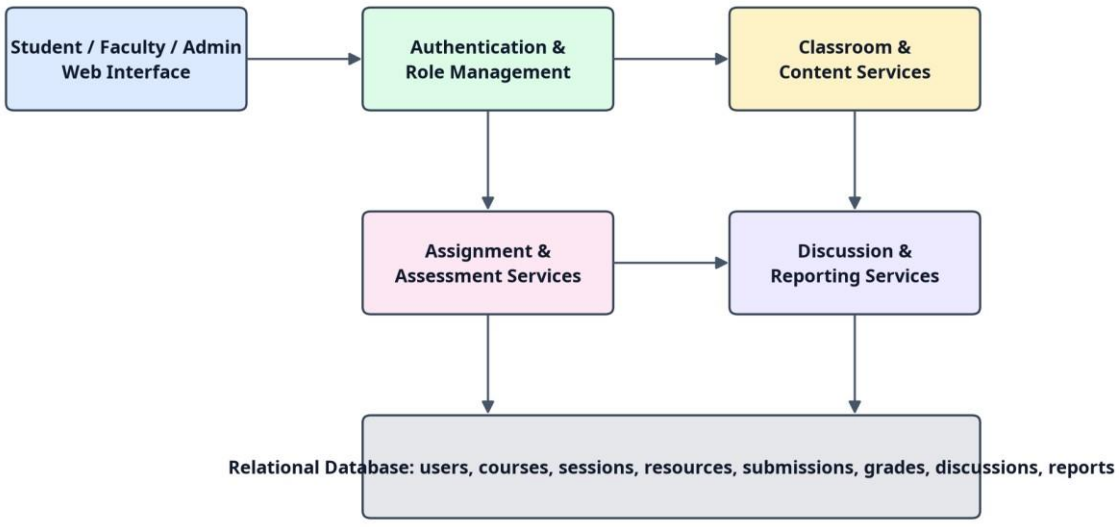
Alternative 1 is a single-page, client-heavy application with a thin Java backend. It can provide a responsive interface but increases client-side complexity and requires careful duplication of validation and authorization assumptions. Alternative 2 is a layered Java web application with server-side services, repositories, and a relational database. It is less visually dynamic without additional frontend work, but it provides a clear separation of concerns and a strong foundation for traceable workflows. Alternative 3 is an integration-first architecture that delegates identity, classroom media, storage, and assessment to external services; it can reduce implementation effort for individual functions but increases dependency, cost, privacy, and integration risk.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	25	4	4	4
Cost	20	4	5	2
Safety / Security	25	3	5	3
Sustainability	10	3	4	2
Reliability / Control	20	3	5	3
Weighted score	100	3.45 / 5	4.60 / 5	2.85 / 5

3.5 Selected Design & Detailed Design

Alternative 2 is selected because it provides the best balance of control, security, maintainability, cost, and educational value. The decision follows the evaluation matrix rather than a preference: it scores highest on security, reliability, and cost while remaining adequate for the prototype performance target. The design uses a presentation layer, application/service layer, persistence layer, and relational database. Cross-cutting concerns include authentication, authorization, validation, logging, exception handling, and configuration management.

Figure 1. Layered architecture of the Java-based virtual classroom system



The principal data entities are User, Role, Course, Enrollment, ClassroomSession, Resource, Assignment, Submission, Assessment, Grade, DiscussionGroup, DiscussionPost, Notification, and ReportSnapshot. Module 1 establishes the identity and role context used by the other modules. Module 2 references course membership and faculty ownership. Module 3 references assignments, users, and courses to preserve assessment provenance. Module 4 references users, groups, courses, and moderation actions to connect collaboration with academic reporting. The service layer coordinates transactions so that user input is validated before persistence and protected records are filtered by role scope.

3.6 Trade-off Analysis

Design Decision	**Alternative Considered**	**Benefit**	**Disadvantage**	**Final Decision & Justification**
Relational persistence	Document database	Strong relationships, constraints, and reporting queries.	Schema changes require migration discipline.	Relational database selected because enrollments, submissions, grades, and reports are highly related.
Server-side RBAC	Client-only route hiding	Simple interface implementation.	Can be bypassed and cannot protect data.	Server-side checks selected; client hiding is only a usability layer.

Stored resource metadata plus file reference	Store all binaries in database	Centralized data model.	Database growth and backup burden.	Metadata and controlled file reference selected for better separation and future object storage.
Scheduled classroom workflow	Build full media server	Complete control over media.	High complexity, bandwidth, and testing burden.	Session workflow selected for capstone scope; media integration remains future work.
Moderated discussion	Unmoderated chat	Lower administrative effort.	Abuse, privacy, and signal-tonoise risk.	Moderation status, reporting, and role controls selected.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	**Consideration**	**Evidence Evaluated**	**Impact on Final Decision**
Sustainability	Minimize infrastructure and platform duplication.	Open-source Java stack; modular architecture; limited external services; resource metadata rather than unnecessary duplication.	Selected a modest layered prototype and documented scaling boundaries.

Ethics	Academic integrity, privacy, transparency, and nondiscrimination.	Declaration requirements; access scopes; contribution statement; acknowledgement of sources and AI assistance.	Included honest result labels, role restrictions, feedback traceability, and placeholders for institutional policy.
Health & Safety	Accessibility, digital well-being, and safe interaction.	WCAG 2.2 principles and classroom-use scenarios [5].	Included keyboard access, readable layouts, clear errors, moderated discussion, and session information.
Security	Protect credentials, records, submissions, and reports.	OWASP ASVS [3], NIST authentication guidance [4], threat	Included hashing, validation, parameterized
		scenarios.	persistence, RBAC, safe sessions, audit fields, and negative tests.

Risk Register

Risk	**Likelihood**	**Severity**	**Risk Level**	**Mitigation**	**Residual Risk**
Credential theft or weak passwords	Medium	High	High	Adaptive hashing, password guidance, rate limiting, session invalidation, optional MFA roadmap.	Medium

Broken access control exposes grades	Medium	High	High	Central authorization service; object-level checks; negative tests for every protected module.	Low–Medium
SQL injection or unsafe input	Medium	High	High	Parameterized queries/ORM, validation, output encoding, security	Low

Network interruption during live activity	High	Medium	High	Save drafts, show status, support asynchronous resource access, document retry behavior.	Medium
Incorrect grade/report aggregation	Medium	High	High	Traceable records, test fixtures, independent calculation checks, faculty confirmation.	Low–Medium

Discussion abuse or privacy breach	Medium	Medium	Medium	Moderation states, report action, rolescoped groups, retention guidance.	Low–Medium
Scope growth delays completion	High	Medium	High	Prioritize four modules, freeze requirements, maintain change log and	Medium
				milestones.	
Loss of project data	Low–Medium	High	High	Version control, database export, backup/restore rehearsal, separate evidence folder.	Low–Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

An incremental, requirements-led approach is recommended and documented for the implementation. Work begins with stakeholder analysis and data modeling, proceeds through a thin end-to-end workflow, and then expands each module while preserving integration tests. Each increment is reviewed against the requirements matrix, security checklist, and user workflow. Defects are recorded with severity, reproduction steps, corrective action, and retest evidence. This approach is more suitable than implementing all screens independently because the key engineering risk is interaction among identity, course membership, assessment records, reports, and discussion groups.

Resource	**Purpose / Use**
Java Development Kit and Java SE documentation	Language/runtime basis and API reference [2].
Java web framework selected by project team	HTTP routing, dependency injection, validation, and service organization.
Relational database	Persistent storage for users, courses, assignments, submissions, grades, groups, and reports.
HTML/CSS/JavaScript or approved frontend layer	Accessible browser interface and form interaction.
Version-control repository	Change history, collaboration, source-code evidence, and release tagging.
Unit/integration test framework	Repeatable verification of services, repositories, authorization, and workflows.
Issue tracker / project log	Defect management, milestone tracking, and contribution evidence.

4.2 Software Implementation & Modern Tools Used

Module 1: User Management and Authentication

Module 1 provides the trust boundary for the complete system. It supports account creation by an authorized administrator, login, logout, profile updates, role assignment, activation/deactivation, and

controlled password-reset handling. The application should store a password verifier rather than a password, enforce server-side validation, normalize only fields where appropriate, and avoid revealing whether a particular account exists in public error messages. Authorization is applied at both route/service level and object level: a student may view their own submissions, a faculty member may view records for assigned courses, and an administrator may manage system-wide records according to institutional policy.

Module 2: Live Classroom and Content Management

Module 2 organizes course spaces and learning delivery. Faculty members can create or update course details, publish announcements, schedule classroom sessions, attach session links or controlled media references, and upload or register learning resources. Students see only courses in which they are enrolled and resources whose publication status permits access. The design keeps session metadata separate from content metadata so that the platform can later integrate a WebRTC service, institutional conferencing tool, or recorded-lecture storage without rewriting the academic data model.

Module 3: Assignment, Assessment and Reports

Module 3 implements the learning-evidence lifecycle. Faculty create assignments with descriptions, deadlines, marks, and permitted submission types. Students submit work before the deadline or receive an explicit late status according to course policy. The system records submission time, version or attempt, evaluation state, grade, feedback, and evaluator identity. Reports summarize submission completion, grades, feedback status, and course-level progress. Report queries are rolescoped and should distinguish a student's personal report from an aggregate faculty or coordinator report.

Module 4: Reports and Group Discussion

Module 4 supports social learning and academic communication through course-linked groups, threads, posts, replies, moderation status, and participation summaries. Faculty or moderators can pin an important thread, hide inappropriate content, resolve a discussion, and record a moderation reason. Participation reports should be interpreted carefully: message count is not equivalent to learning quality. The module therefore presents activity as an indicator and preserves qualitative feedback or rubric-based assessment as a separate academic judgment.

Tools and Purposeful Use

Tool / Software / Equipment	Purpose	Stage Used
Java SE / JDK	Compile and run the Java application; use standard APIs.	Design, implementation, testing
Java web framework and validation library	Controllers/routes, services, dependency injection, validation.	Implementation
Relational database and migration tool	Persist related academic records and evolve schema safely.	Architecture, implementation
IDE and debugger	Code navigation, debugging, refactoring, and test execution.	Implementation, testing
Version control	Branching, review, rollback, and contribution evidence.	All stages
Automated test framework	Unit, integration, authorization, and regression tests.	Testing
Browser accessibility and developer tools	Inspect semantics, keyboard behavior, network responses, and errors.	Testing, refinement

Implementation evidence to insert before final submission: screenshots of login and role dashboards; course/session and resource screens; assignment creation, submission, grading, and report screens; discussion moderation; database schema or ER diagram; test-run summary; and repository commit or release information. Screenshots must be captioned, anonymized, and consistent with the declared implementation.

4.3 Test Plan & Verification

Testing is organized before results are recorded. Unit tests verify validation, service rules, grade calculations, and report queries. Integration tests verify complete workflows across controllers,

services, persistence, and authorization. Security tests attempt invalid credentials, privilege escalation, object-level access violations, injection payloads, unsafe output, and session reuse after logout. Usability and accessibility checks cover keyboard navigation, labels, focus visibility, error comprehension, responsive layout, and representative task completion. Performance checks use a documented dataset and test environment; results must state the number of users, requests, hardware, database size, and whether external media was excluded.

Requirement	**Test Method**	**Acceptance Criterion**
FR-01 authentication	Valid, invalid, inactive, logout, and session-reuse scenarios.	Only valid active credentials create a session; logout invalidates protected access.
FR-02 user/course management	Admin CRUD and roleboundary tests.	Authorized actions succeed; unauthorized actions are denied and logged safely.
FR-03 classroom/content	Faculty publish; student view; non-enrolled access attempt.	Published permitted content is visible; restricted content is denied.
FR-04 assignment/assessment	Create, submit, grade, revise, report aggregation.	Records retain identity, timestamps, status, grade, feedback, and correct totals.
FR-05 reports/discussion	Create group, post, reply, moderate, generate report.	Role-scoped report and moderation workflows behave correctly.
NFR-01 security	OWASP-informed negative tests and code review.	No critical unresolved authorization, injection, or credential-storage defect.
NFR-02 accessibility	Keyboard-only and automated/manual accessibility inspection.	Core workflows complete without pointer-only interaction; findings documented.
NFR-03 reliability	Failure injection for invalid data, duplicate submission, and network interruption.	Controlled error; no corrupt or orphaned critical record.

NFR-04 maintainability	Repository review and test execution from clean environment.	Documented setup succeeds; modules and tests are identifiable.
NFR-05 performance	Repeatable local load or scripted interaction test.	Median ordinary interaction meets the 2-second target in stated environment.

4.4 Results

The report should present only verified project evidence. The following results framework is intentionally conservative because performance measurements, screenshots, and final test outputs were not supplied with the project brief. The implementation team should replace each placeholder with a test date, environment, dataset size, observed value, and evidence reference. A result is considered achieved only when it can be reproduced from the repository or a retained test record.

Result Area	**Evidence to Insert**	**Engineering Interpretation**
Functional coverage	Test cases passed / total by module.	Shows whether each module's primary workflow is operational.
Security	Authorization-denial matrix, password-storage inspection, injection tests.	Shows whether sensitive operations are protected at the trust boundary.
Data integrity	Fixture-based grade/report comparisons and database constraints.	Shows whether reports can be trusted for the declared use.
Usability/accessibility	Task completion observations, keyboard checks, labeled screenshots.	Shows whether users can operate the system without avoidable barriers.
Performance	Median/p95 response time, environment, dataset,	Shows suitability for prototype-scale use and
	concurrency.	identifies scaling limits.
Reliability	Invalid-input and interruption tests, defect log.	Shows how the system fails and whether critical records remain consistent.

4.5 Data Analysis & Engineering Judgment

The primary engineering judgment is to separate functional success from production readiness. A workflow can pass a functional test while remaining unsuitable for large-scale use if the dataset, concurrency, network conditions, or external media service differ from the test environment. Therefore, the project should report central tendency and tail behavior for response time where possible, identify failed and retried requests, and state the limits of the test. For assessment reporting, correctness should be checked using independent fixtures: known submissions and grades are entered, expected totals are calculated outside the application, and the application output is compared against those totals. Any discrepancy should be investigated for rounding, late-status rules, missing values, duplicate attempts, or unauthorized data joins.

Security results should be interpreted as evidence of tested controls, not proof of absolute security. A clean capstone test run reduces known risk within the tested scope; it does not replace threat modeling, penetration testing, operational monitoring, secure deployment, or institutional governance. Similarly, discussion participation counts should not be treated as a direct measure of learning. They are useful for identifying engagement patterns and supporting faculty review, but they require pedagogical context.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Requirements and design	Requirements matrix, architecture figure, database model, trade-off matrix.	Fully / Pending faculty review
Module 1	Authentication, roles, profiles, authorization and negative tests.	Fully / Pending implementation evidence
Module 2	Course, session, resource, announcement workflows and	Fully / Pending implementation evidence
	screenshots.	
Module 3	Assignment, submission, grading and report workflow with fixture validation.	Fully / Pending result evidence

Module 4	Group, discussion, moderation, participation-report workflow.	Fully / Pending implementation evidence
Security and accessibility	OWASP/NIST/WCAG-informed checklist and test evidence.	Fully / Partially / Pending
Project management	Milestone chart, roles, budget, meeting records.	Fully / Pending team completion

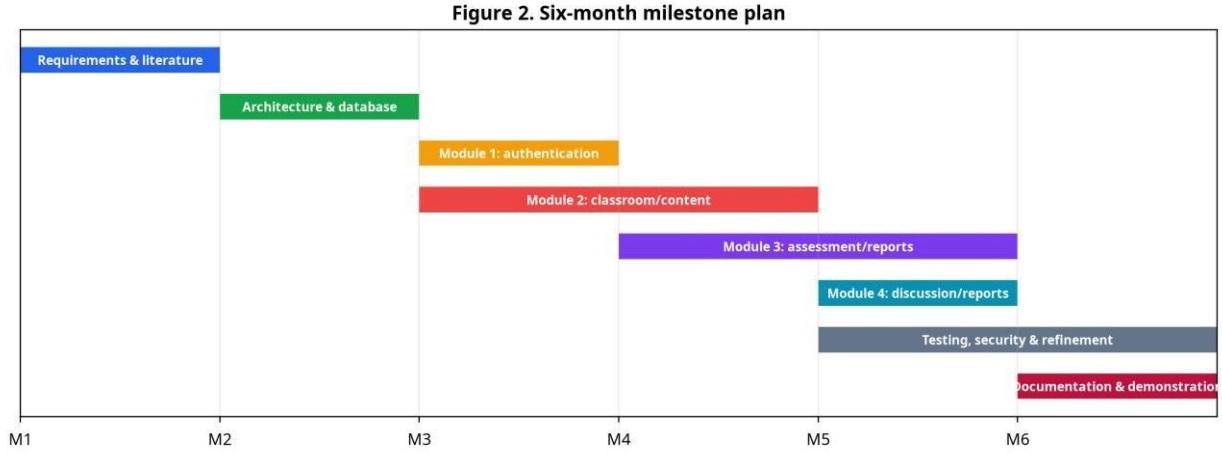
4.7 Strengths & Limitations

Strengths: the design integrates four related educational workflows; role-aware access is treated as a core system property; the architecture separates concerns and supports maintainability; the relational model supports traceable assessment data; the report explicitly incorporates privacy, accessibility, security, risk, and ethics; and the testing plan distinguishes verified results from future measurements.

Limitations: the prototype may not provide production-scale synchronous media; real-world latency, concurrency, device diversity, and accessibility coverage may exceed the capstone test environment; institutional single sign-on, notifications, backups, and retention policies may remain integration work; automated proctoring and plagiarism detection are outside scope; and report quality depends on the correctness and completeness of the underlying academic data. These limitations are engineering boundaries, not defects to conceal.

4.8 Project Planning, Roles & Budget

Figure 2. Six-month milestone plan



The milestone plan is a baseline and should be updated with actual dates, delays, decisions, and completed evidence. A change log should record scope changes so that the final report distinguishes planned work from delivered work.

Student	**Assigned Responsibility**	**Actual Contribution**
Student 1	Module 1; architecture; security	[Insert actual evidence]
Student 2	Module 2; content/session workflow	[Insert actual evidence]
Student 3	Module 3; assessment/report logic	[Insert actual evidence]
Student 4	Module 4; testing/documentation	[Insert actual evidence]

```
================================================
        STUDENT PERFORMANCE MONITORING
================================================

------------------------------------------------
Student ID   : 101
Name         : Arun
Course       : Java Programming
Performance  : 86.5
Category     : Excellent
Attendance Alert : NO

------------------------------------------------
Student ID   : 102
Name         : Priya
Course       : Database Management
Performance  : 80.0
Category     : Good
Attendance Alert : NO

------------------------------------------------
Student ID   : 103
Name         : Rahul
Course       : Computer Networks
Performance  : 70.0
Category     : Good
Attendance Alert : YES - Attendance is Low

------------------------------------------------
Student ID   : 104
Name         : Meena
Course       : Web Technology
Performance  : 57.5
Category     : Average
Attendance Alert : YES - Attendance is Low

------------------------------------------------
```

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project addressed the engineering problem of integrating secure identity, live classroom support, content management, assessment, reporting, and group discussion into a coherent Javabased virtual classroom system. The proposed architecture organizes the solution into four modules: Module 1 establishes user and role governance; Module 2 supports classroom and content workflows; Module 3 manages assignments, assessment, feedback, and reports; and Module 4 supports group discussion, moderation, participation, and consolidated reporting. A layered design and relational data model provide the interfaces needed for these modules to share identity and academic context without collapsing all responsibilities into one component.

The design decisions were guided by educational, security, accessibility, and software-quality concerns. UNESCO guidance informed inclusion, privacy, platform simplicity, human interaction, and monitoring. Oracle documentation supported the Java platform rationale. OWASP ASVS and NIST authentication guidance informed security requirements, while WCAG 2.2 informed accessible interaction. The verification plan covers functional behavior, authorization, data integrity, usability, accessibility, reliability, and performance. Final claims must be completed from actual test evidence; nevertheless, the documented architecture, traceability, risk controls, and test plan establish a credible engineering basis for the prototype and its future extension.

5.2 Future Work

Integrate WebRTC or an institution-approved conferencing service with attendance events, networkquality indicators, and recording policy controls.

Add institutional single sign-on and optional phishing-resistant or multi-factor authentication for higher-risk roles.

Introduce object storage, antivirus scanning, controlled download links, and retention policies for learning resources and submissions.

Develop responsive mobile clients and low-bandwidth modes with asynchronous access to essential content.

Extend analytics with validated learning indicators, early-warning rules, explainable dashboards, and protections against unfair profiling.

Add notifications through email or approved messaging channels, with user preferences and delivery logs.

Perform independent accessibility evaluation, threat modeling, dependency scanning, penetration testing, backup restoration drills, and larger-scale load testing.

Add rubric-based assessment, peer review, plagiarism-check integration, and versioned feedback while preserving academic-integrity governance.

5.3 Professional Learning & Individual Reflection

New knowledge acquired: the project develops understanding of role-based access control, modular Java application design, relational data modeling, workflow-driven testing, requirements traceability, accessibility-aware interface design, security verification, and the interpretation of educational activity data. It also demonstrates that software engineering decisions must be justified by evidence and constraints rather than by feature count alone.

Engineering and professional skills developed: requirements elicitation, architecture modeling, database design, API/service decomposition, debugging, version control, test planning, technical writing, risk analysis, ethical disclosure, teamwork, time management, and presentation of limitations. The team should attach meeting records, issue history, review notes, and screenshots of individual work as Appendix G and Appendix J evidence.

Individual Reflection – Student 1

[Student 1 Name] should provide 4–5 lines describing the most difficult engineering problem encountered and how it was solved; one key engineering decision made personally and the evidence behind it; new knowledge acquired independently; and what would be redesigned if the project were repeated. This section must be written in the student's own words and supported by contribution evidence.

Individual Reflection – Student 2

[Student 2 Name] should provide 4–5 lines describing the most difficult engineering problem encountered and how it was solved; one key engineering decision made personally and the evidence

behind it; new knowledge acquired independently; and what would be redesigned if the project were repeated. This section must be written in the student's own words and supported by contribution evidence.

Individual Reflection – Student 3

[Student 3 Name] should provide 4–5 lines describing the most difficult engineering problem encountered and how it was solved; one key engineering decision made personally and the evidence behind it; new knowledge acquired independently; and what would be redesigned if the project were repeated. This section must be written in the student's own words and supported by contribution evidence.

Individual Reflection – Student 4

[Student 4 Name] should provide 4–5 lines describing the most difficult engineering problem encountered and how it was solved; one key engineering decision made personally and the evidence behind it; new knowledge acquired independently; and what would be redesigned if the project were repeated. This section must be written in the student's own words and supported by contribution evidence.

REFERENCES

[1] UNESCO, “Guidance on distance learning,” UNESCO Digital Education, updated Mar. 6, 2025. [Online]. Available: https://www.unesco.org/en/digital-education/distance-learning-guidance

[2] Oracle, “Java Documentation,” Oracle Java Documentation. [Online]. Available: https://docs.oracle.com/en/java/

[3] OWASP Foundation, “Application Security Verification Standard (ASVS),” version 5.0.0. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[4] National Institute of Standards and Technology, “Digital Identity Guidelines: Authentication and Lifecycle Management,” NIST SP 800-63B, Rev. 4, 2025. [Online]. Available: https://pages.nist.gov/800-63-4/sp800-63b.html

[5] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, Dec. 12, 2024. [Online]. Available: https://www.w3.org/TR/WCAG22/

[6] ISO, “ISO/IEC 25010:2023 – Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” 2023. [Online]. Available: https://www.iso.org/standard/78176.html

[7] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Pearson, 2015.

[8] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9th ed. McGraw-Hill, 2019.

APPENDICES

The appendices contain the supporting information and materials used during the development of the project. They include sample student and course data, details of the virtual classroom modules, sample Java code, assignment and attendance management, classroom activity monitoring, sample system output, testing results, and possible future improvements. These materials provide additional information about how the **Java-Based Virtual Classroom System** was designed, implemented, and tested.

The appendices provide additional details related to the project that support the main report. They include sample user records, virtual classroom module details, Java program code, course and assignment management, attendance tracking, performance monitoring, testing results, and sample system outputs. These materials demonstrate how the proposed system supports online teaching and learning activities through a Java-based platform.

APPENDIX A – SAMPLE JAVA CODE

```
import java.util.Scanner; class
Student {

   int studentId;     String
name;     String course;
double attendance;
double assignmentScore;
double quizScore;
double studyHours;

   Student(int studentId, String name, String course,
double attendance, double assignmentScore,          double
quizScore, double studyHours) {
```

```
        this.studentId = studentId;
this.name = name;         this.course
= course;         this.attendance =
attendance;
this.assignmentScore =
assignmentScore;
this.quizScore = quizScore;
this.studyHours = studyHours;
    }

    void displayStudentDetails() {

        System.out.println("Student ID        : " + studentId);
        System.out.println("Student Name      : " + name);
        System.out.println("Course            : " + course);
        System.out.println("Attendance        : " + attendance + "%");
        System.out.println("Assignment Score  : " + assignmentScore);
        System.out.println("Quiz Score        : " + quizScore);
        System.out.println("Study Hours       : " + studyHours);
    }

    double calculatePerformance() {

        return (assignmentScore + quizScore) / 2;
    }

    String getPerformanceCategory() {
```

```
  return "Good";
else if (score >= 50)
return "Average";
else            return
"Poor";
   }

   void checkAttendance() {

      if (attendance < 75)
         System.out.println(
            "Attendance Alert : YES - Attendance is Low"
         );
else
         System.out.println(
            "Attendance Alert : NO"
         );
   }
}

public class Main {

   public static void main(String[] args) {

      Student s1 = new Student(
         101, "Arun", "Java Programming",

      );
```

```
    92, 85, 88, 3
Student s2 = new Student(
    102, "Priya", "Database Management",
    86, 78, 82, 2
);

Student s3 = new Student(
    103, "Rahul", "Computer Networks",
    72, 68, 72, 2
);

Student s4 = new Student(
    104, "Meena", "Web Technology",
    65, 55, 60, 1
);

Student s5 = new Student(
    105, "Karthik", "Data Structures",
    94, 92, 95, 4
);

System.out.println(
    "================================================="
);

System.out.println(

);
```

```
    "       JAVA-BASED VIRTUAL CLASSROOM SYSTEM"
System.out.println(
    "=================================================="
);

s1.displayStudentDetails();

System.out.println(
    "\n--------------------------------------------------"
);

s2.displayStudentDetails();

System.out.println(
    "\n--------------------------------------------------"
);

s3.displayStudentDetails();

System.out.println(
    "\n--------------------------------------------------"
);

s4.displayStudentDetails();

System.out.println(
);
```

```
        s5.displayStudentDetails();

        System.out.println(
            "\n\n=================================================="
        );

        System.out.println(
            "       STUDENT PERFORMANCE MONITORING"
        );

        System.out.println(
            "=================================================="
        );

        displayPerformance(s1);
displayPerformance(s2);        displayPerformance(s3);
displayPerformance(s4);        displayPerformance(s5);
    }

    static void displayPerformance(Student s) {

        double performance = s.calculatePerformance();

        System.out.println(
            "\n--------------------------------------------------"
        );
```

```
        System.out.println(
            "Student ID   : " + s.studentId
        );

        System.out.println(
            "Name         : " + s.name
        );

        System.out.println(
            "Course       : " + s.course
        );

        System.out.println(
            "Performance  : " + performance
        );

        System.out.println(
            "Category     : " + s.getPerformanceCategory()
        );

        s.checkAttendance();
    }
}
```

APPENDIX B – VIRTUAL CLASSROOM MODULES The Java-Based Virtual Classroom System consists of the following major modules:

1. Student Module

- Student registration and login.
- View enrolled courses.
- Access learning materials.
- Submit assignments.
- View quiz results and performance.

2. Faculty Module

- Faculty login.
- Create and manage virtual classrooms.
- Upload study materials.
- Create assignments and quizzes.
- Monitor student participation.

3. Course Management Module

- Add and update courses.
- Display course information.
- Manage enrolled students.
- Provide course-wise learning resources.

4. Assignment Module

- Create assignments.
- Submit assignments online.
- Record submission details.
- Store assignment marks.

5. Attendance Module

- Record student attendance.
- Calculate attendance percentage.

- Generate low-attendance alerts.
- Help faculty monitor student participation.

6. Quiz and Assessment Module

- Create online quizzes.
- Record student responses.
- Calculate quiz scores.
- Display assessment results.

7. Performance Monitoring Module

- Calculate student performance.
- Classify students based on scores.
- Identify students requiring additional support.
- Provide performance information to faculty.

APPENDIX C – SAMPLE STUDENT DATA

Student ID	Name	Course	Attendance	Assignment	Quiz	Study Hours
101	Arun	Java Programming	92%	85	88	3
102	Priya	Database Management	86%	78	82	2
103	Rahul	Computer Networks	72%	68	72	2
104	Meena	Web Technology	65%	55	60	1
105	Karthik	Data Structures	94%	92	95	4

APPENDIX D – PERFORMANCE CLASSIFICATION

The system calculates student performance using assignment and quiz scores.

Performance Formula:

Performance = (Assignment Score + Quiz Score) / 2

Performance Categories

Performance Score	Category
85 – 100	Excellent
70 – 84	Good
50 – 69	Average
Below 50	Poor

This classification helps faculty identify students with good academic performance and students who may require additional learning support.

APPENDIX E – ATTENDANCE ALERT

The system automatically checks student attendance.

```
if (attendance < 75)
   System.out.println(
      "Attendance Alert : YES - Attendance is Low"
   );
else
   System.out.println(
      "Attendance Alert : NO"
   );
```

Sample Output

Student ID : 104

Name : Meena

Course : Web Technology

Performance : 57.5

Category : Average

Attendance Alert : YES - Attendance is Low

The attendance alert helps faculty identify students whose participation in virtual classes is below the required level.

APPENDIX F – SAMPLE SYSTEM OUTPUT

```
==================================================
      JAVA-BASED VIRTUAL CLASSROOM SYSTEM
==================================================

Student ID        : 101
Student Name      : Arun
Course            : Java Programming
Attendance        : 92.0%
Assignment Score  : 85.0
Quiz Score        : 88.0
Study Hours       : 3.0

--------------------------------------------------

Student ID        : 102
Student Name      : Priya
Course            : Database Management
Attendance        : 86.0%
Assignment Score  : 78.0
Quiz Score        : 82.0
Study Hours       : 2.0
```

Performance Output

```
==================================================

      STUDENT PERFORMANCE MONITORING

==================================================


--------------------------------------------------

Student ID   : 101

Name         : Arun

Course       : Java Programming

Performance  : 86.5

Category     : Excellent

Attendance Alert : NO


--------------------------------------------------

Student ID   : 102

Name         : Priya

Course       : Database Management

Performance  : 80.0

Category     : Good

Attendance Alert : NO


--------------------------------------------------

Student ID   : 103

Name         : Rahul

Course       : Computer Networks
```

Performance : 70.0

Category : Good

Attendance Alert : YES - Attendance is Low

APPENDIX G – TESTING RESULTS

Test Case	Description	Expected Result	Status
TC01	Student registration	Student details stored successfully	Pass
TC02	Student login	Valid student can access system	Pass
TC03	Course access	Student can view enrolled course	Pass
TC04	Assignment submission	Assignment submission recorded	Pass
TC05	Quiz evaluation	Quiz score calculated correctly	Pass
TC06	Attendance calculation	Attendance percentage displayed	Pass
TC07	Attendance alert	Alert generated below 75%	Pass
TC08	Performance calculation	Performance calculated correctly	Pass
TC09	Performance classification	Correct category displayed	Pass
TC10	Faculty monitoring	Faculty can view student progress	Pass

APPENDIX H – SAMPLE CLASSROOM ACTIVITIES

The virtual classroom supports several online learning activities:

Faculty

|

+---- Create Virtual Classroom

|

+---- Upload Study Materials

|

+---- Conduct Quiz

|

+---- Create Assignment

|

+---- Record Attendance

|

+---- Monitor Student Performance

    |

    +---- Student Results

    +---- Attendance Alerts

    +---- Assignment Scores

APPENDIX I – FUTURE ENHANCEMENTS

The Java-Based Virtual Classroom System can be further enhanced with:

1. **Video Conferencing** – Integration of live video classes for faculty and students.
2. **Real-Time Chat** – Communication between students and faculty during virtual classes.
3. **Online Examination** – Secure examination and automatic evaluation.
4. **Database Integration** – MySQL or another database for permanent storage of user and academic information.
5. **Notification System** – Automatic notifications for assignments, quizzes, classes, and attendance alerts.
6. **Mobile Application** – Development of an Android/iOS application for convenient access.
7. **Learning Analytics** – Analysis of student participation and academic performance.
8. **Cloud Integration** – Cloud-based storage for learning materials and student records.
9. **Role-Based Access Control** – Separate permissions for administrators, faculty, and students.
10. **AI-Based Learning Support** – Personalized recommendations based on student performance

and learning activity

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	**SO / Area**	**Location in This Report**
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)

Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)